

JAVA PROGRAMMING

COMPREHENSIVE INSTRUCTOR MANUAL

Foundations to Mastery

Instructor: Mr. Jefferson Youashi

OSS Developer

Topics 1 – 4 | Foundations & Core Concepts

TOPIC 1

Java Foundations & Environment Setup

1.1 What is Java?

Java is a high-level, object-oriented, class-based programming language designed with a primary goal: write once, run anywhere (WORA). Originally developed by James Gosling at Sun Microsystems in 1995, Java compiles source code into platform-neutral bytecode that runs on the Java Virtual Machine (JVM). Today it powers enterprise backends, Android apps, financial systems, cloud services, and embedded devices.

Core Characteristics

- **Platform Independence:** Compiled to bytecode, not native machine code. The JVM interprets or JIT-compiles bytecode on any supported OS.
- **Object-Oriented:** Everything is modeled as objects with state (fields) and behavior (methods).
- **Strongly Typed:** Every variable must have a declared type; the compiler enforces type rules at compile time.
- **Automatic Memory Management:** The Garbage Collector (GC) reclaims unused objects; no manual malloc/free.
- **Multi-threaded:** Built-in support for concurrent execution through threads and the `java.util.concurrent` package.
- **Rich Standard Library:** The Java SE API spans thousands of classes covering I/O, networking, data structures, security, and more.
- **Robust & Secure:** Strong exception handling, type safety, bytecode verification, and a security manager prevent common errors and exploits.

Primary Use Cases

Domain	Examples & Frameworks
Enterprise Back-end	Spring Boot REST APIs, EJB, Jakarta EE microservices
Android Development	Android SDK, Jetpack Compose (via Kotlin/Java interop)
Web Applications	Servlets, JSP, Spring MVC, Play Framework
Financial Systems	High-frequency trading platforms (low-latency Java)
Big Data	Apache Hadoop, Apache Spark, Apache Kafka
Cloud Services	AWS Lambda (Java runtime), Google Cloud Functions
Embedded & IoT	Java ME, embedded controllers
Desktop GUIs	JavaFX, Swing

1.2 JDK vs JRE vs JVM — The Java Ecosystem

Beginners often confuse these three acronyms. Understanding their relationship is fundamental before writing any code.

Component	Purpose & Contents
JVM — Java Virtual Machine	The runtime engine that executes Java bytecode. Performs class loading, bytecode verification, JIT compilation, garbage collection. Platform-specific (Windows JVM ≠ Linux JVM).
JRE — Java Runtime Environment	JVM + standard class libraries (java.lang, java.util, java.io ...). Sufficient to run pre-compiled Java programs.
JDK — Java Development Kit	JRE + development tools: javac (compiler), javadoc (docs generator), jdb (debugger), jar (archiver), jshell (REPL). Required to compile and develop Java applications.

Key Takeaway

For development you always install the JDK. The JRE alone cannot compile code. Modern JDK distributions (OpenJDK 17, 21) bundle everything.

The Compilation & Execution Pipeline

Understanding this pipeline explains how your .java file becomes a running program:

Source File		Compiler		Bytecode		JVM		
HelloWorld.java	→	javac	→	HelloWorld.class	→	java	→	Output

1. You write Java source code (.java)
2. javac compiles it to bytecode (.class) - platform-neutral
3. java (JVM) loads .class, verifies, and executes bytecode
4. JIT compiler optimises hot code paths at runtime

JIT Compilation

The Just-In-Time compiler inside the JVM detects frequently executed bytecode ('hot spots') and compiles them to native machine code at runtime, giving Java near-native performance for long-running programs.

1.3 Installing JDK & Setting Up Your IDE

Installing OpenJDK 21 (LTS)

OpenJDK 21 is the current long-term support release. Download from <https://adoptium.net> (Eclipse Temurin) or <https://jdk.java.net>.

1. Download the installer for your OS (Windows .msi / macOS .pkg / Linux .tar.gz).
2. Run the installer with default settings.
3. Verify installation — open a terminal and run:

```
java -version
javac -version
```

4. Set JAVA_HOME environment variable to your JDK installation directory (e.g., C:\Program Files\Java\jdk-21).
5. Add %JAVA_HOME%\bin (Windows) or \$JAVA_HOME/bin (Unix) to your PATH.

IDE Setup — IntelliJ IDEA (Recommended)

6. Download IntelliJ IDEA Community Edition (free) from <https://www.jetbrains.com/idea/>
7. Install and launch. Choose New Project → Java.
8. Set the Project SDK to your JDK 21 installation.
9. Enable auto-import and set the project language level to 21.
10. Useful shortcuts: Shift+F10 (Run), Ctrl+/ (Comment), Alt+Enter (Quick Fix), Ctrl+Space (Autocomplete).

IDE Setup — Eclipse (Alternative)

11. Download Eclipse IDE for Java Developers from <https://www.eclipse.org/downloads/>
12. Extract and launch eclipse.exe. Select a workspace folder.
13. File → New → Java Project. Enter project name, select JDK 21 as JRE.
14. Right-click src → New → Class to create your first class.

1.4 Your First Java Program — Hello World!

By convention, every Java programmer's first program prints 'Hello, World!' to the console. Here is the complete source:

```
// File: HelloWorld.java
public class HelloWorld {

    public static void main(String[] args) {
        System.out.println("Hello, World!");
    }
}
```

Line-by-Line Anatomy

Element	Explanation
public class HelloWorld	Declares a class named HelloWorld. In Java, every piece of code lives inside a class. The filename must match the class name exactly (HelloWorld.java).
public	Access modifier — the class/method is visible from anywhere.
static	Belongs to the class itself, not to an instance. The JVM can call main() without creating an object.
void	Return type. main() does not return a value.
main(String[] args)	The program entry point. args holds command-line arguments passed at startup.
System.out.println(...)	Prints text followed by a newline. System is a class, out is a static PrintStream field, println() is a method.

Compiling and Running from the Terminal

```
# Compile
javac HelloWorld.java

# Run
java HelloWorld

# Expected output:
Hello, World!
```

Common Beginner Mistake

Java is case-sensitive. 'helloworld', 'HelloWorld', and 'HELLOWORLD' are three different identifiers. The class name and the filename (without .java) must match exactly including capitalisation.

1.5 Java Syntax Basics — Statements, Blocks & Comments

Statements

A statement is a complete unit of execution in Java. Every statement ends with a semicolon (;). Examples:

```
int x = 10;
System.out.println(x);
double result = x * 2.5;
```

Blocks

A block is a group of zero or more statements enclosed in curly braces { }. Blocks define scope and group related statements:

```
// Block example — defines a scope
{
    int a = 5;
    int b = 10;
    System.out.println(a + b);
}
// a and b are out of scope here
```

Comments

Comments are ignored by the compiler. Use them to document your intent, not to re-describe the code:

```
// Single-line comment: from // to end of line

/* Multi-line comment
   spans several lines
   use sparingly           */

/**
 * Javadoc comment — generates HTML API documentation.
 * @param name the name to greet
```

```

    * @return      the greeting string
    */
    public String greet(String name) { ... }

```

💡 Good Commenting Practice

Comments should explain WHY something is done, not WHAT is done (the code itself shows what). Outdated comments are worse than no comments — always keep them in sync with code changes.

1.6 Primitive Data Types

Java has eight primitive types. They are stored directly in memory (on the stack), not as objects. They are the building blocks of all data in Java.

Type	Size	Range / Values	Default	Use Case
<code>byte</code>	8-bit	-128 to 127	0	Raw binary, small integer
<code>short</code>	16-bit	-32,768 to 32,767	0	Conserving memory in large arrays
<code>int</code>	32-bit	-2^{31} to $2^{31}-1$	0	General-purpose integer (most common)
<code>long</code>	64-bit	-2^{63} to $2^{63}-1$	0L	Large numbers, timestamps, IDs
<code>float</code>	32-bit	$\pm 3.4 \times 10^{38}$ (7 sig digits)	0.0f	Memory-constrained floating-point
<code>double</code>	64-bit	$\pm 1.8 \times 10^{308}$ (15 sig digs)	0.0	Scientific, financial calculations
<code>char</code>	16-bit	0 to 65,535 (Unicode)	'\u0000'	Single character / Unicode code point
<code>boolean</code>	1-bit	true / false	false	Flags, conditions, control flow

Literals and Suffixes

```

int    count    = 1_000_000;    // underscore for readability (Java 7+)
long   bigNumber = 9_876_543_210L; // L suffix required
float  price     = 9.99f;        // f suffix required
double pi        = 3.14159265;   // default for decimal literals
char   letter    = 'A';          // single quotes, one character only
char   unicode   = '\u0041';     // same as 'A' in Unicode escape
boolean isActive = true;         // only true or false
int     hex      = 0xFF;         // hexadecimal literal
int     binary   = 0b1010_1010; // binary literal (Java 7+)

```

⚠️ float vs double Precision

Avoid float for financial calculations. float has only ~7 significant digits; double has ~15. For money, use `java.math.BigDecimal` to avoid rounding errors.

1.7 Variable Declaration, Initialisation & Naming Conventions

Declaration and Initialisation

A variable is a named storage location. In Java, you must declare the type before use. Initialisation assigns a value.

```
// Declaration (no initial value — may cause compile error if read before
assignment)
int age;

// Declaration + Initialisation
int age = 25;
String name = "Alice";

// final — value cannot change after first assignment (Java constant)
final double TAX_RATE = 0.075;

// var — local variable type inference (Java 10+)
var message = "Hello"; // inferred as String
```

Java Naming Conventions

Category	Convention & Example
Variables	camelCase — age, firstName, totalPrice
Methods	camelCase — calculateTotal(), getUserName()
Classes	PascalCase (UpperCamelCase) — BankAccount, StudentRecord
Constants	ALL_UPPER_SNAKE — MAX_SIZE, TAX_RATE, PI
Packages	all lowercase, domain-reversed — com.company.project
Interfaces	PascalCase, often adjective — Runnable, Serializable



Why Conventions Matter

Consistent naming makes code self-documenting. A variable named 'd' is far less readable than 'dailyInterestRate'. Adhere to conventions — they are enforced by code review in professional teams.

1.8 Console Output with System.out & Input with Scanner

Output Methods

```
// println() — prints and moves to new line
System.out.println("Hello, Java!");

// print() — prints WITHOUT newline
System.out.print("No newline");

// printf() — formatted output (like C's printf)
double price = 29.99;
System.out.printf("Price: $%.2f%n", price);
```

```
// String.format() — build formatted string without printing
String msg = String.format("Age: %d, Name: %s", 25, "Bob");

// Text Block (Java 15+) — multi-line string literal
String json = """
    {
        "name": "Alice",
        "age": 30
    }
    """;
```

Format Specifiers Quick Reference

Specifier	Meaning & Example
%d	Decimal integer — printf("%d", 42) → 42
%f	Floating-point — printf("%.2f", 3.14159) → 3.14
%s	String — printf("%s", "Java") → Java
%c	Character — printf("%c", 'A') → A
%b	Boolean — printf("%b", true) → true
%n	Platform-independent newline (preferred over \n in printf)
%10d	Right-align in field width 10
%-10d	Left-align in field width 10
%05d	Zero-pad to width 5 — printf("%05d", 42) → 00042

Reading Input with Scanner

```
import java.util.Scanner;

public class InputDemo {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        System.out.print("Enter your name: ");
        String name = scanner.nextLine();

        System.out.print("Enter your age: ");
        int age = scanner.nextInt();

        System.out.print("Enter your GPA: ");
        double gpa = scanner.nextDouble();

        System.out.printf("Hello, %s! Age: %d, GPA: %.2f%n", name, age,
gpa);
        scanner.close();
    }
}
```


Scanner Method Reference

Method	What It Reads
<code>nextLine()</code>	Reads entire line including spaces until newline
<code>next()</code>	Reads single token (stops at whitespace)
<code>nextInt()</code>	Reads next integer token
<code>nextDouble()</code>	Reads next double token
<code>nextLong()</code>	Reads next long token
<code>nextBoolean()</code>	Reads next boolean token (true/false)
<code>hasNextInt()</code>	Returns true if next token is a valid int
<code>hasNextLine()</code>	Returns true if another line exists in input

⚠ `nextInt()` + `nextLine()` Pitfall

After `nextInt()`, the newline character remains in the buffer. Calling `nextLine()` immediately after reads that empty newline. Fix: call `scanner.nextLine()` once after `nextInt()` to consume the leftover newline before reading a string.

TOPIC 2

Operators, Control Flow & Conditionals

2.1 Arithmetic, Relational & Logical Operators

Arithmetic Operators

```
int a = 17, b = 5;

System.out.println(a + b); // 22 - addition
System.out.println(a - b); // 12 - subtraction
System.out.println(a * b); // 85 - multiplication
System.out.println(a / b); // 3 - integer division (truncates)
System.out.println(a % b); // 2 - modulo (remainder)

// Integer division truncates - NOT rounds!
System.out.println(7 / 2); // 3 (not 3.5)
System.out.println(7.0 / 2); // 3.5 (one operand is double)

// Compound assignment operators (shorthand)
a += 3; // a = a + 3
a -= 2; // a = a - 2
a *= 4; // a = a * 4
a /= 3; // a = a / 3
a %= 2; // a = a % 2

// Increment and decrement
a++; // post-increment: use a, then a = a + 1
++a; // pre-increment: a = a + 1, then use a
```

Relational (Comparison) Operators

Relational operators always produce a boolean result (true or false). They are the foundation of all conditional logic.

```
int x = 10, y = 20;

System.out.println(x == y); // false - equal to
System.out.println(x != y); // true - not equal to
System.out.println(x < y); // true - less than
System.out.println(x > y); // false - greater than
System.out.println(x <= y); // true - less than or equal
System.out.println(x >= y); // false - greater than or equal
```

Logical Operators

Logical operators combine boolean expressions. Java uses short-circuit evaluation: if the result is determined by the left operand, the right operand is NOT evaluated.

```
boolean a = true, b = false;

System.out.println(a && b); // false - AND: both must be true
System.out.println(a || b); // true - OR: at least one true
```

```

System.out.println(!a);      // false — NOT: inverts the value

// Short-circuit evaluation example
int n = 0;
if (n != 0 && 10 / n > 2) { // 10/n is NEVER evaluated when n==0
    System.out.println("Safe");
}

```

Bitwise Operators (Advanced Reference)

Operator	Meaning & Example
&	Bitwise AND — 5 & 3 = 1 (0101 & 0011 = 0001)
	Bitwise OR — 5 3 = 7 (0101 0011 = 0111)
^	Bitwise XOR — 5 ^ 3 = 6 (0101 ^ 0011 = 0110)
~	Bitwise NOT — ~5 = -6
<<	Left shift — 5 << 1 = 10 (multiply by 2)
>>	Right shift — 20 >> 2 = 5 (divide by 4)
>>>	Unsigned right shift — fills with 0 regardless of sign

2.2 Operator Precedence & Associativity

Operator precedence determines the order in which operations are evaluated in an expression. Associativity determines the direction when operators have equal precedence.

Level	Operators	Description	Associativity
1 (high)	() [] . ++ -- (postfix)	Postfix / grouping	Left to Right
2	++ -- + - ~ ! (prefix)	Prefix / unary	Right to Left
3	* / %	Multiplicative	Left to Right
4	+ -	Additive	Left to Right
5	<< >> >>>	Shift	Left to Right
6	< <= > >= instanceof	Relational	Left to Right
7	== !=	Equality	Left to Right
8	&	Bitwise AND	Left to Right
9	^	Bitwise XOR	Left to Right
10		Bitwise OR	Left to Right
11	&&	Logical AND	Left to Right
12		Logical OR	Left to Right
13	?:	Ternary	Right to Left
14 (low)	= += -= *= /= %= ...	Assignment	Right to Left

```
// Precedence in action — trace this mentally:
int result = 2 + 3 * 4 - 1; // = 2 + 12 - 1 = 13 (* before +/-)
int r2      = (2 + 3) * (4 - 1); // = 5 * 3 = 15 (parens override)
boolean b   = 5 > 3 && 2 < 8 || false; // true && true || false = true
```

2.3 Type Casting — Implicit and Explicit

Widening (Implicit) Casting

Java automatically widens a smaller type to a larger compatible type. No data is lost. The conversion happens silently.

```
// Widening hierarchy: byte → short → int → long → float → double
int i = 100;
long l = i; // int → long (automatic)
double d = i; // int → double (automatic)
System.out.println(d); // 100.0
```

Narrowing (Explicit) Casting

Converting from a larger type to a smaller one may lose data. You must explicitly cast — this is your declaration to the compiler that you accept the risk.

```
double pi = 3.14159;
int truncated = (int) pi; // 3 — decimal part lost

int big = 300;
byte small = (byte) big; // 44 — overflow/wrapping!

// String ↔ primitive conversions
String s = "42";
int n = Integer.parseInt(s); // String → int
double x = Double.parseDouble(s); // String → double
String back = String.valueOf(n); // int → String
String back2 = Integer.toString(n); // int → String (alternative)
```

2.4 Conditional Statements

if / if-else / else-if Ladder

```
int score = 78;
String grade;

if (score >= 90) {
    grade = "A";
} else if (score >= 80) {
    grade = "B";
} else if (score >= 70) {
    grade = "C";
} else if (score >= 60) {
```

```
    grade = "D";
} else {
    grade = "F";
}

System.out.println("Grade: " + grade);
```

Nested if & Complex Boolean Expressions

```
int age = 22;
boolean hasID = true;
boolean isMember = false;

if (age >= 18 && hasID) {
    if (isMember) {
        System.out.println("Welcome, VIP member!");
    } else {
        System.out.println("Welcome, guest!");
    }
} else {
    System.out.println("Access denied.");
}
```

2.5 switch Statement & Enhanced switch (Java 14+)

Traditional switch

The traditional switch checks a variable against a series of constant values. Without break, execution 'falls through' to the next case — a common source of bugs.

```
int day = 3;
switch (day) {
    case 1: System.out.println("Monday");    break;
    case 2: System.out.println("Tuesday");   break;
    case 3: System.out.println("Wednesday"); break;
    case 4:
    case 5: System.out.println("Weekday");    break;
    default: System.out.println("Weekend");
}
```

Enhanced switch Expression (Java 14+) — Preferred Style

The new switch expression eliminates fall-through, removes the need for break, and can return a value directly. Use this in modern Java code.

```
// Enhanced switch as expression — returns a value
String dayName = switch (day) {
    case 1 -> "Monday";
    case 2 -> "Tuesday";
    case 3 -> "Wednesday";
    case 4, 5 -> "Weekday";
    default -> "Weekend";
}
```

```
};

// Multi-line case with yield
int numLetters = switch (dayName) {
    case "Monday", "Friday", "Sunday" -> yield 6;
    case "Saturday" -> { yield 8; }
    default -> yield 0;
};
```

2.6 Ternary Operator

The ternary operator is a concise one-liner for simple if-else assignments. Syntax: condition ? valueIfTrue : valueIfFalse

```
int age = 20;
String status = (age >= 18) ? "Adult" : "Minor";
System.out.println(status); // Adult

// Nested ternary (use sparingly — prefer if-else for readability)
int score = 75;
String grade = score >= 90 ? "A"
    : score >= 80 ? "B"
    : score >= 70 ? "C" : "F";

// Ternary in expressions
int max = (a > b) ? a : b;
int abs = (n >= 0) ? n : -n;
```

Lab 2 — Decision-Making Programs

Exercise Set A: Grade Calculator

- Exercise 1. Write a program that accepts a student's numeric score (0–100) and prints the letter grade (A, B, C, D, F) using an if-else ladder. Also print 'Excellent!' if A, 'Good' if B.
- Exercise 2. Extend Exercise 1: accept scores for 5 subjects, compute the average, then assign and print the final letter grade for the average.
- Exercise 3. A university uses this grading scale: 90–100 = A+, 85–89 = A, 80–84 = B+, 75–79 = B, 70–74 = C+, 65–69 = C, 60–64 = D, below 60 = F. Implement a program with this scale.
- Exercise 4. Add validation to Exercise 1: if the score is below 0 or above 100, print 'Invalid score. Please enter a value between 0 and 100.' using a nested if inside the main logic.
- Exercise 5. Rewrite Exercise 1 using the ternary operator for the A/not-A check, and an if-else chain for the rest.
- Exercise 6. Write a program that reads a student's percentage and determines both grade AND pass/fail status. Passing requires at least 50%. Output: 'Grade: B — PASS'.



Exercise Set B: Simple Menu System

- Exercise 7. Build a text-based calculator menu. Display: 1) Add 2) Subtract 3) Multiply 4) Divide 5) Exit. Read two numbers and an operation choice, then display the result. Use a switch statement.
- Exercise 8. Extend Exercise 7: add input validation — if the user selects Divide and enters 0 as the divisor, print 'Error: Division by zero is undefined.' instead of crashing.
- Exercise 9. Write a program that reads a month number (1–12) and prints: the month name, whether it belongs to Q1/Q2/Q3/Q4, and the number of days (assume non-leap year). Use an enhanced switch.
- Exercise 10. Create a simple ATM simulation menu: 1) Check Balance 2) Deposit 3) Withdraw 4) Exit. Initialise balance at 5000.00. Validate that withdrawals do not exceed balance.
- Exercise 11. Write a program that accepts a character ('A'–'Z') and classifies it as Vowel or Consonant using a switch expression with multiple case labels.
- Exercise 12. Build a shipping cost calculator: the user enters a weight in kg and selects a shipping type (1=Standard, 2=Express, 3=Overnight). Rates: Standard \$2/kg, Express \$5/kg, Overnight \$10/kg plus a \$15 flat fee. Use nested if and switch.
- Exercise 13. Implement a BMI classifier: read weight (kg) and height (m), compute $BMI = \text{weight}/\text{height}^2$. Classify: <18.5 Underweight, 18.5–24.9 Normal, 25–29.9 Overweight, ≥30 Obese.



Exercise Set C: Challenge Problems

- Exercise 14. Write a program that reads three integers and determines: the maximum, the minimum, and whether they can form the sides of a valid triangle (sum of any two sides > third side).
- Exercise 15. Implement a simplified tax calculator: income below \$10,000 — no tax; \$10,000–\$50,000 — 15%; above \$50,000 — 30% on the excess over \$50,000 plus \$6,000. Print the taxable amount.
- Exercise 16. Write a leap year checker: a year is a leap year if divisible by 4, EXCEPT century years (divisible by 100) which must also be divisible by 400. Test: 2000 (leap), 1900 (not), 2024 (leap).

TOPIC 3

Loops, Iteration & Problem Solving

3.1 The while Loop — Condition-Controlled Iteration

The while loop repeats a block as long as its condition remains true. The condition is checked BEFORE each iteration. If the condition is false initially, the body never executes.

```
// Syntax:
while (condition) {
    // body executed repeatedly
}

// Example 1: count from 1 to 5
int i = 1;
while (i <= 5) {
    System.out.print(i + " ");
    i++;
} // Output: 1 2 3 4 5

// Example 2: sum digits of a number
int number = 12345, sum = 0;
while (number > 0) {
    sum += number % 10; // extract last digit
    number /= 10;      // remove last digit
}
System.out.println("Sum: " + sum); // Sum: 15
```

Infinite Loop Guard

Always ensure the loop variable is modified inside the body to eventually make the condition false. Forgetting `i++` (or equivalent) creates an infinite loop that freezes your program.

3.2 The do-while Loop — Guaranteed First Execution

The do-while loop evaluates the condition AFTER each iteration. This guarantees the body executes at least once — essential for input validation menus.

```
// Syntax:
do {
    // body always executes at least once
} while (condition); // note the semicolon!

// Classic use case: input validation
import java.util.Scanner;
Scanner sc = new Scanner(System.in);
int choice;
do {
    System.out.print("Enter 1-5: ");
    choice = sc.nextInt();
} while (choice < 1 || choice > 5);
```



```
System.out.println("Valid: " + choice);
```

3.3 The for Loop — Counter-Controlled Iteration

The for loop is ideal when the number of iterations is known in advance. All three control parts (init; condition; update) are in one line for clarity.

```
// Syntax:
for (initialisation; condition; update) {
    // body
}

// Count 1 to 10
for (int i = 1; i <= 10; i++) {
    System.out.print(i + " ");
}

// Sum 1 to N using accumulator pattern
int n = 100, total = 0;
for (int i = 1; i <= n; i++) {
    total += i;
} // total = 5050

// Count down — decrement
for (int i = 10; i >= 1; i--) {
    System.out.print(i + " ");
} // 10 9 8 7 6 5 4 3 2 1

// Step by 2 — only even numbers
for (int i = 0; i <= 20; i += 2) {
    System.out.print(i + " ");
}
```

3.4 The Enhanced for-each Loop

The for-each loop provides a clean syntax to iterate over arrays and collections without managing an index. It is read-only — you cannot modify elements via the loop variable.

```
// Syntax:
for (elementType element : collection) {
    // use element
}

int[] numbers = {10, 20, 30, 40, 50};
int sum = 0;
for (int num : numbers) {
    sum += num;
}

String[] fruits = {"Apple", "Banana", "Cherry"};
```

```
for (String fruit : fruits) {  
    System.out.println(fruit);  
}
```

3.5 Nested Loops — Patterns, Tables & Shapes

A nested loop has one loop inside another. The inner loop completes all its iterations for each single iteration of the outer loop. Use nested loops for 2D patterns and matrix operations.

```
// Multiplication table (1-5 x 1-5)  
for (int i = 1; i <= 5; i++) {  
    for (int j = 1; j <= 5; j++) {  
        System.out.printf("%4d", i * j);  
    }  
    System.out.println(); // move to next row  
}  
  
// Right-aligned star triangle:  
// *  
// **  
// ***  
int rows = 5;  
for (int i = 1; i <= rows; i++) {  
    for (int j = 1; j <= i; j++) {  
        System.out.print("*");  
    }  
    System.out.println();  
}
```

3.6 Loop Control — break and continue

```
// break — immediately exits the enclosing loop  
for (int i = 1; i <= 10; i++) {  
    if (i == 6) break;  
    System.out.print(i + " "); // prints 1 2 3 4 5  
}  
  
// continue — skips current iteration, jumps to next  
for (int i = 1; i <= 10; i++) {  
    if (i % 2 == 0) continue; // skip even numbers  
    System.out.print(i + " "); // prints 1 3 5 7 9  
}  
  
// Labeled break — exits specific outer loop  
outerLoop:  
for (int i = 0; i < 5; i++) {  
    for (int j = 0; j < 5; j++) {  
        if (i + j == 6) break outerLoop;  
        System.out.println(i + "," + j);  
    }  
}
```

3.7 Common Loop Patterns

Accumulator Pattern

Use a variable (accumulator) that collects a running total across iterations.

```
// Sum of first N natural numbers
int n = 10, sum = 0;
for (int i = 1; i <= n; i++) sum += i;
System.out.println("Sum: " + sum); // 55
```

Counter Pattern

Count occurrences of a condition within a dataset.

```
// Count how many numbers in 1-100 are divisible by 7
int count = 0;
for (int i = 1; i <= 100; i++) {
    if (i % 7 == 0) count++;
} // count = 14
```

Sentinel-Controlled Pattern

Loop continues until the user enters a special terminating value (the sentinel).

```
import java.util.Scanner;
Scanner sc = new Scanner(System.in);
int total = 0, count = 0;
System.out.println("Enter scores (-1 to stop):");
int input = sc.nextInt();
while (input != -1) {
    total += input;
    count++;
    input = sc.nextInt();
}
if (count > 0) System.out.printf("Average: %.2f\n", (double) total / count);
```



Lab 3 — Loops & Iterative Problem Solving



Exercise Set A: Number Series

Exercise 17. Print all odd numbers between 1 and 50 using a for loop. Count and display how many there are.

Exercise 18. Print the Fibonacci sequence up to the 20th term. ($F(1)=1$, $F(2)=1$, $F(n)=F(n-1)+F(n-2)$). Display each term on a new line with its position.

Exercise 19. Print the first 15 powers of 2: 2^1 , 2^2 , ..., 2^{15} . Use a while loop with a running product variable.

Exercise 20. Write a program that reads a number N and prints its multiplication table from 1 to 12 in formatted output: '7 x 3 = 21'.

Exercise 21. Print all prime numbers between 2 and 100. A prime has exactly two divisors: 1 and itself. Use nested loops.

Exercise 22. Read a positive integer and print its reverse. Example: 12345 $\rightarrow$ 54321. Use modulo and integer division in a while loop.

Exercise 23. Collatz Conjecture: start with any positive integer n . If n is even, divide by 2; if odd, multiply by 3 and add 1. Count how many steps it takes to reach 1.

Exercise Set B: Validation and Shapes

Exercise 24. Write an input validation loop that keeps prompting until the user enters a positive integer less than 1000. Print 'Valid input received: X' once accepted.

Exercise 25. Print a hollow rectangle of asterisks of dimensions $R \times C$ (read from the user). Only the border should have stars; the interior should be spaces.

Exercise 26. Print a diamond pattern of stars with a user-specified middle width (must be odd). The pattern should expand and then contract symmetrically.

Exercise 27. Write a menu-driven program that repeatedly shows options (1=Rectangle area, 2=Triangle area, 3=Circle area, 4=Exit) and computes the selected area. Use a do-while loop for the menu.

Exercise 28. Implement a number guessing game: generate a random number between 1 and 100 (use `Math.random()` or `Random`). Allow the user up to 7 guesses. Print 'Too high', 'Too low', or 'Correct!'. Show the number of attempts used.

Exercise 29. Read 10 integers from the user. After input, print: the maximum, the minimum, the sum, the average, and how many were above the average.

Exercise Set C: Challenge Problems

Exercise 30. Write a program to check if a number is a perfect number. A perfect number equals the sum of its proper divisors (e.g., $6 = 1+2+3$). Print all perfect numbers up to 10,000.

Exercise 31. Implement a digital root calculator: repeatedly sum the digits of a number until a single digit remains. Example: $9875 \rightarrow 9+8+7+5=29 \rightarrow 2+9=11 \rightarrow 1+1=2$.

Exercise 32. Write a program that prints Pascal's Triangle up to N rows (read from user). Compute each row using the binomial coefficient formula.

TOPIC 4

Methods: Modularity and Reusability

4.1 Defining and Calling Methods

A method is a named, reusable block of code that performs a specific task. Methods eliminate code duplication, improve readability, and isolate logic so it can be tested independently.

```
// General syntax:
[accessModifier] [static] returnType methodName(parameterList) {
    // method body
    return value; // omit if return type is void
}

// Concrete example
public class MethodDemo {

    // Method definition
    public static int add(int a, int b) {
        return a + b;
    }

    public static void greet(String name) {
        System.out.println("Hello, " + name + "!");
    }

    public static void main(String[] args) {
        int result = add(10, 25); // calling add()
        System.out.println(result); // 35
        greet("Alice");           // Hello, Alice!
    }
}
```

4.2 Method Signatures — Return Types, Parameters & void

A method signature uniquely identifies a method: its name and the types/order of its parameters. The return type is part of the declaration but NOT part of the signature for overloading purposes.

Component	Details
Return Type	The type of value the method returns. Use void if nothing is returned. The compiler enforces that every code path returns a value of the declared type.
Method Name	Follows camelCase conventions. Should be a verb or verb-phrase describing what the method does: calculateTax(), isValid(), getUserId().
Parameters	Local variables that receive values from the caller. A method can have 0 or more parameters. They are comma-separated: (int a, double b, String name).

Component	Details
return Statement	Immediately exits the method and optionally sends a value back. A void method can use a bare 'return;' to exit early.
Varargs (...)	Allows a variable number of arguments of the same type: void print(String... lines). Internally treated as an array.

```
// void method — no return value
public static void printDivider() {
    System.out.println("-".repeat(40));
}

// multiple return points — allowed but use carefully
public static String classify(int n) {
    if (n > 0) return "positive";
    if (n < 0) return "negative";
    return "zero";
}

// varargs example
public static int sum(int... numbers) {
    int total = 0;
    for (int n : numbers) total += n;
    return total;
}

// call with any number of args:
sum(1, 2, 3);    // 6
sum(1, 2, 3, 4, 5); // 15
```

4.3 Pass-by-Value in Java

Java is strictly pass-by-value. When you pass a primitive to a method, a copy of the value is made. Modifying the parameter inside the method has NO effect on the original variable.

```
public static void tryToModify(int x) {
    x = 999; // modifies local copy only
}

int original = 42;
tryToModify(original);
System.out.println(original); // still 42!

// Swap example — doesn't work with primitives!
public static void swap(int a, int b) {
    int temp = a; a = b; b = temp; // no effect outside
}
```

Object References are Also Passed by Value

When you pass an object, the reference (memory address) is copied — not the object itself. So you CAN modify the object's contents via the reference, but you CANNOT make the caller's variable point to a different object.

4.4 Method Overloading

Method overloading allows multiple methods with the same name but different parameter lists (type, count, or order). The compiler determines which version to call at compile time — this is static polymorphism.

```
public class Calculator {

    // Overload 1: two ints
    public static int add(int a, int b) {
        return a + b;
    }

    // Overload 2: three ints
    public static int add(int a, int b, int c) {
        return a + b + c;
    }

    // Overload 3: two doubles
    public static double add(double a, double b) {
        return a + b;
    }

    public static void main(String[] args) {
        System.out.println(add(3, 4));           // 7    - calls overload 1
        System.out.println(add(1, 2, 3));        // 6    - calls overload 2
        System.out.println(add(1.5, 2.5));        // 4.0  - calls overload 3
    }
}
```

⚠ Overloading vs Overriding

Overloading (same class, different signatures) is resolved at compile time. Overriding (subclass redefines a method) is resolved at runtime via polymorphism — covered in the OOP topics.

4.5 Variable Scope — Local, Instance & Static

Scope Type	Where Declared / Lifetime / Accessibility
Local Variable	Inside a method or block. Created when the block is entered; destroyed when exited. Cannot be accessed outside the block. MUST be initialised before use.
Instance Variable (Field)	Inside a class but outside any method. One copy per object instance. Exists as long as the object exists. Accessible from all non-static methods of the class.
Class Variable (Static)	Declared with 'static' keyword inside a class. One copy shared by ALL instances. Exists for the lifetime of the class. Accessible via <code>ClassName.fieldName</code> .

Scope Type	Where Declared / Lifetime / Accessibility
Block Variable	Declared inside a for/while/if block. Scope is strictly the enclosing braces.

```

public class ScopeDemo {
    static int classCount = 0;           // class (static) variable
    int instanceId;                     // instance variable

    ScopeDemo() {
        classCount++;
        instanceId = classCount;
    }

    public static void main(String[] args) {
        int localVar = 10;             // local variable
        ScopeDemo obj1 = new ScopeDemo();
        ScopeDemo obj2 = new ScopeDemo();
        System.out.println(classCount); // 2
        System.out.println(obj1.instanceId); // 1
        System.out.println(obj2.instanceId); // 2
    }
}

```

4.6 Recursive Methods — Concept & Examples

A recursive method calls itself to solve a smaller version of the same problem. Every recursive method needs a base case (stops recursion) and a recursive case (reduces the problem).

```

// Factorial: n! = n × (n-1)!
// Base case: 0! = 1, 1! = 1
public static long factorial(int n) {
    if (n <= 1) return 1L;           // base case
    return n * factorial(n - 1);     // recursive case
}

// Call stack trace for factorial(4):
// factorial(4) = 4 × factorial(3)
//              = 4 × 3 × factorial(2)
//              = 4 × 3 × 2 × factorial(1)
//              = 4 × 3 × 2 × 1 = 24

// Fibonacci: F(n) = F(n-1) + F(n-2)
public static int fib(int n) {
    if (n <= 1) return n;
    return fib(n - 1) + fib(n - 2);
}

// Power: a^n = a × a^(n-1)
public static double power(double base, int exp) {
    if (exp == 0) return 1;
    return base * power(base, exp - 1);
}

```



```

}

// GCD using Euclidean algorithm
public static int gcd(int a, int b) {
    return (b == 0) ? a : gcd(b, a % b);
}

```

⚠ Stack Overflow Risk

Deep recursion without a reachable base case causes a `StackOverflowError` — the call stack runs out of memory. The naive recursive Fibonacci is also extremely slow for large n (exponential time). For large inputs, prefer iterative or memoized solutions.

4.7 Java's Math Library & Standard Utility Methods

The `java.lang.Math` class provides static methods for common mathematical operations. It is imported automatically — no import statement needed.

Method	Description & Example
<code>Math.abs(x)</code>	Absolute value — <code>Math.abs(-7) = 7</code>
<code>Math.max(a, b)</code>	Larger of two values — <code>Math.max(10, 25) = 25</code>
<code>Math.min(a, b)</code>	Smaller of two values — <code>Math.min(3, 8) = 3</code>
<code>Math.pow(base, exp)</code>	Power — <code>Math.pow(2, 10) = 1024.0</code>
<code>Math.sqrt(x)</code>	Square root — <code>Math.sqrt(144) = 12.0</code>
<code>Math.cbrt(x)</code>	Cube root — <code>Math.cbrt(27) = 3.0</code>
<code>Math.ceil(x)</code>	Round up — <code>Math.ceil(3.2) = 4.0</code>
<code>Math.floor(x)</code>	Round down — <code>Math.floor(3.9) = 3.0</code>
<code>Math.round(x)</code>	Round to nearest integer — <code>Math.round(3.5) = 4</code>
<code>Math.log(x)</code>	Natural logarithm (base e)
<code>Math.log10(x)</code>	Base-10 logarithm
<code>Math.sin/cos/tan(r)</code>	Trigonometric functions (argument in radians)
<code>Math.toRadians(deg)</code>	Convert degrees to radians
<code>Math.random()</code>	Pseudo-random double in $[0.0, 1.0)$
<code>Math.PI</code>	The constant $\pi \approx 3.14159265358979$
<code>Math.E</code>	Euler's number $e \approx 2.71828182845904$

```

// Practical Math examples
double hypotenuse = Math.sqrt(Math.pow(3, 2) + Math.pow(4, 2)); // 5.0
int randInt = (int) (Math.random() * 100) + 1; // 1-100
double circleArea = Math.PI * Math.pow(radius, 2);
long rounded = Math.round(Math.PI * 1000) / 1000.0;

```



Lab 4 — Methods & Multi-Method Calculation Utility



Exercise Set A: Basic Method Writing

- Exercise 33. Write a method `boolean isEven(int n)` that returns true if `n` is even, false otherwise. Call it from `main()` for 10 test values and print results.
- Exercise 34. Write a method `int clamp(int value, int min, int max)` that returns value clamped to `[min, max]`. Example: `clamp(150, 0, 100) = 100`.
- Exercise 35. Write a method `double celsiusToFahrenheit(double c)` and `double fahrenheitToCelsius(double f)`. Print a conversion table from -20°C to 100°C in 10° increments.
- Exercise 36. Write a method `String repeatString(String s, int times)` that returns `s` concatenated times times. Example: `repeatString("ab", 3) = "ababab"`.
- Exercise 37. Write a method `boolean isPalindrome(String s)` that returns true if `s` reads the same forwards and backwards (ignore case). Test with 'Racecar', 'hello', 'Madam'.
- Exercise 38. Write a method `int countVowels(String s)` that returns the number of vowels in a string (a, e, i, o, u — both upper and lower case).



Exercise Set B: Method Overloading

- Exercise 39. Overload a method `area()` for three shapes: `area(double radius)` for circle, `area(double base, double height)` for triangle, `area(double length, double width)` for rectangle.
- Exercise 40. Overload a method `print()` to accept `int`, `double`, `String`, and `boolean` parameters. Each version should print the value with its type label: 'int: 42', 'double: 3.14', etc.
- Exercise 41. Overload `max()` to find the maximum of 2 ints, 3 ints, 2 doubles, and 3 doubles.
- Exercise 42. Create a method `power()` with overloads: `power(int base, int exp)`, `power(double base, int exp)`, and `power(double base, double exp)` using `Math.pow()`.



Exercise Set C: Recursive Methods

- Exercise 43. Implement `factorial(int n)` recursively and iteratively. Run both for `n = 0` to 15 and verify both versions produce identical results.
- Exercise 44. Implement a recursive method `int sumDigits(int n)` that returns the sum of digits of `n`. Example: `sumDigits(12345) = 15`.
- Exercise 45. Implement a recursive method `String reverse(String s)` that returns the reversed string.
- Exercise 46. Implement a recursive method `boolean isPrime(int n, int divisor)` — start the call with `isPrime(n, 2)`. Return false if `n % divisor == 0` and `divisor < n`; return true if `divisor == n`.
- Exercise 47. Implement recursive binary search: `int binarySearch(int[] arr, int target, int low, int high)`. Return the index of target or -1 if not found.
- Exercise 48. Implement the Tower of Hanoi: `void hanoi(int n, char from, char to, char via)`. Print each move. Count total moves for `n = 1` to 10.



Exercise Set D: Multi-Method Calculation Utility (Capstone Lab)

- Exercise 49. Build a complete MathUtility class with methods: gcd(int a, int b) using Euclidean recursion, lcm(int a, int b) using gcd, isPrime(int n), nextPrime(int n) (next prime after n), nthFibonacci(int n) iteratively.
- Exercise 50. Build a Statistics class with methods: mean(double[] data), median(double[] data) — sort and pick middle, mode(int[] data) — most frequent value, range(double[] data), variance(double[] data), standardDeviation(double[] data).
- Exercise 51. Build a GeometryCalculator with overloaded methods: perimeter and area for circle, rectangle, triangle, square, and trapezoid. Use Math.PI and Math.sqrt as needed.
- Exercise 52. Build a UnitConverter class with method chains: length (mm, cm, m, km, in, ft, mile), temperature (C, F, K), weight (g, kg, lb, oz). Each method converts from a base unit to target.
- Exercise 53. Build an interactive menu-driven app that ties together MathUtility, Statistics, and GeometryCalculator. The menu shows categories; the user selects a function, provides input, and sees results. Loop until exit.
- Exercise 54. Build a LoanCalculator class with methods: monthlyPayment(double principal, double annualRate, int months), totalInterest(double principal, double annualRate, int months), amortizationTable(double principal, double annualRate, int months) — prints month-by-month balance, interest paid, and principal paid.
- Exercise 55. Write a StringUtils class with methods: capitalize(String s) — capitalise each word, countWords(String s), longestWord(String s), anagramCheck(String a, String b) — true if same characters, caesarCipher(String text, int shift) — encode/decode.